

SQL QUERIES GUIDE

Complete SQL Commands for DBMS

DDL | DML | Aggregate & Scalar Functions

Prepared: March 14, 2026

TABLE OF CONTENTS

- 1. DDL (Data Definition Language) Queries
 - 1.1 CREATE TABLE
 - 1.2 ALTER TABLE
 - 1.3 DROP TABLE
 - 1.4 TRUNCATE TABLE
- 2. DML (Data Manipulation Language) Queries
 - 2.1 INSERT
 - 2.2 UPDATE
 - 2.3 DELETE
 - 2.4 SELECT
- 3. Aggregate Functions
 - 3.1 COUNT, SUM, AVG
 - 3.2 MAX, MIN
 - 3.3 GROUP BY, HAVING
- 4. Scalar Functions
 - 4.1 String Functions
 - 4.2 Numeric Functions
 - 4.3 Date Functions
- 5. Query Examples with Practical Scenarios

1. DDL (DATA DEFINITION LANGUAGE) QUERIES

DDL statements are used to define and manage database structure. They include CREATE, ALTER, DROP, and TRUNCATE.

1.1 CREATE TABLE - Create New Table

Syntax:

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    ...  
    PRIMARY KEY (column_name)  
);
```

Example 1: Create STUDENT Table

```
CREATE TABLE STUDENT (  
    ROLL_NO INT PRIMARY KEY,  
    NAME VARCHAR(50) NOT NULL,  
    ADDRESS VARCHAR(100),  
    PHONE BIGINT,  
    AGE INT,  
    BRANCH_CODE VARCHAR(10)  
);
```

Example 2: Create BRANCH Table with Foreign Key

```
CREATE TABLE BRANCH (  
    BRANCH_CODE VARCHAR(10) PRIMARY KEY,  
    BRANCH_NAME VARCHAR(50) NOT NULL,  
    HOD_NAME VARCHAR(50),  
    LOCATION VARCHAR(100)  
);
```

Example 3: Add Foreign Key Constraint

```
CREATE TABLE STUDENT (  
    ROLL_NO INT PRIMARY KEY,  
    NAME VARCHAR(50) NOT NULL,  
    BRANCH_CODE VARCHAR(10),  
    FOREIGN KEY (BRANCH_CODE) REFERENCES BRANCH(BRANCH_CODE)  
);
```

1.2 ALTER TABLE - Modify Table Structure

Add Column:

```
ALTER TABLE STUDENT ADD COLUMN EMAIL VARCHAR(50);
```

Modify Column:

```
ALTER TABLE STUDENT MODIFY COLUMN NAME VARCHAR(100);
```

Drop Column:

```
ALTER TABLE STUDENT DROP COLUMN EMAIL;
```

Add Constraint:

```
ALTER TABLE STUDENT ADD CONSTRAINT fk_branch
```

```
FOREIGN KEY (BRANCH_CODE) REFERENCES BRANCH(BRANCH_CODE);
```

Drop Constraint:

```
ALTER TABLE STUDENT DROP CONSTRAINT fk_branch;
```

1.3 DROP TABLE - Delete Entire Table

Delete table and its structure:

```
DROP TABLE STUDENT;
```

Drop with constraint handling:

```
DROP TABLE STUDENT CASCADE CONSTRAINTS;
```

1.4 TRUNCATE TABLE - Delete All Records

Delete all data but keep structure (faster than DELETE):

```
TRUNCATE TABLE STUDENT;
```

Difference from DELETE:

- TRUNCATE is faster
- TRUNCATE cannot be rolled back in some DBMSs
- DELETE can have WHERE clause, TRUNCATE deletes all

2. DML (DATA MANIPULATION LANGUAGE) QUERIES

DML statements manipulate data within tables. They include INSERT, UPDATE, DELETE, and SELECT.

2.1 INSERT - Add New Records

Insert single row with all columns:

```
INSERT INTO STUDENT VALUES (1, 'RAM', 'DELHI', 9455123451, 18, 'CS');
```

Insert with specific columns:

```
INSERT INTO STUDENT (ROLL_NO, NAME, BRANCH_CODE)
VALUES (2, 'RAMESH', 'IT');
```

Insert multiple rows:

```
INSERT INTO STUDENT VALUES
(3, 'SUJIT', 'ROHTAK', 9156253131, 20, 'ECE'),
(4, 'SURESH', 'DELHI', NULL, 18, 'IT'),
(5, 'HARSH', 'PUNE', 9999999999, 19, 'CS');
```

Insert from another table:

```
INSERT INTO STUDENT_BACKUP
SELECT * FROM STUDENT WHERE BRANCH_CODE = 'CS';
```

2.2 UPDATE - Modify Existing Records

Update single column for all records:

```
UPDATE STUDENT SET AGE = 20;
```

Update with WHERE clause:

```
UPDATE STUDENT SET AGE = 21 WHERE ROLL_NO = 1;
```

Update multiple columns:

```
UPDATE STUDENT SET AGE = 22, ADDRESS = 'GURGAON'
WHERE ROLL_NO = 2;
```

Update with condition:

```
UPDATE STUDENT SET BRANCH_CODE = 'CSE'
WHERE BRANCH_CODE = 'CS';
```

Update using calculation:

```
UPDATE STUDENT SET AGE = AGE + 1
WHERE BRANCH_CODE = 'IT';
```

2.3 DELETE - Remove Records

Delete all records (use with caution):

```
DELETE FROM STUDENT;
```

Delete specific record:

```
DELETE FROM STUDENT WHERE ROLL_NO = 5;
```

Delete with complex condition:

```
DELETE FROM STUDENT WHERE AGE < 18 AND BRANCH_CODE = 'CS';
```

Delete records from one table based on another:

```
DELETE FROM STUDENT WHERE BRANCH_CODE IN
  (SELECT BRANCH_CODE FROM BRANCH WHERE LOCATION = 'DELHI');
```

2.4 SELECT - Retrieve Data

Select all columns and rows:

```
SELECT * FROM STUDENT;
```

Select specific columns:

```
SELECT ROLL_NO, NAME, BRANCH_CODE FROM STUDENT;
```

Select with WHERE clause:

```
SELECT * FROM STUDENT WHERE BRANCH_CODE = 'CS';
```

Select with multiple conditions (AND):

```
SELECT * FROM STUDENT
WHERE BRANCH_CODE = 'CS' AND AGE > 18;
```

Select with OR condition:

```
SELECT * FROM STUDENT
WHERE BRANCH_CODE = 'CS' OR BRANCH_CODE = 'IT';
```

Select with IN operator:

```
SELECT * FROM STUDENT
WHERE BRANCH_CODE IN ('CS', 'IT', 'ECE');
```

Select with LIKE (pattern matching):

```
SELECT * FROM STUDENT WHERE NAME LIKE 'R%'; -- Names starting with R
SELECT * FROM STUDENT WHERE NAME LIKE '%SH'; -- Names ending with SH
SELECT * FROM STUDENT WHERE NAME LIKE '%AM%'; -- Names containing AM
```

Select with BETWEEN:

```
SELECT * FROM STUDENT WHERE AGE BETWEEN 18 AND 20;
```

Select with ORDER BY (Sorting):

```
SELECT * FROM STUDENT ORDER BY NAME; -- Ascending
SELECT * FROM STUDENT ORDER BY AGE DESC; -- Descending
```

Select with DISTINCT (Remove Duplicates):

```
SELECT DISTINCT BRANCH_CODE FROM STUDENT;
```

Select with LIMIT (Top N records):

```
SELECT * FROM STUDENT LIMIT 5; -- First 5 records
SELECT * FROM STUDENT LIMIT 5 OFFSET 10; -- Skip first 10, get next 5
```

Select using JOIN (Combining tables):

```
SELECT STUDENT.NAME, BRANCH.BRANCH_NAME
FROM STUDENT
INNER JOIN BRANCH ON STUDENT.BRANCH_CODE = BRANCH.BRANCH_CODE;
```

3. AGGREGATE FUNCTIONS

Aggregate functions operate on multiple rows and return a single value.

3.1 COUNT, SUM, AVG Functions

COUNT - Count number of records:

```
SELECT COUNT(*) AS total_students FROM STUDENT;  
SELECT COUNT(PHONE) AS students_with_phone FROM STUDENT; -- Ignores NULL  
SELECT COUNT(DISTINCT BRANCH_CODE) AS branches FROM STUDENT;
```

SUM - Sum of numeric values:

```
SELECT SUM(MARKS) AS total_marks FROM STUDENTS;  
SELECT SUM(MARKS) FROM STUDENTS WHERE BRANCH_CODE = 'CS';
```

AVG - Average of numeric values:

```
SELECT AVG(AGE) AS average_age FROM STUDENT;  
SELECT AVG(MARKS) AS avg_marks FROM STUDENTS WHERE BRANCH_CODE = 'IT';
```

3.2 MAX and MIN Functions

MAX - Maximum value:

```
SELECT MAX(AGE) AS max_age FROM STUDENT;  
SELECT MAX(MARKS) AS highest_marks FROM STUDENTS;  
SELECT NAME, MARKS FROM STUDENTS WHERE MARKS = (SELECT MAX(MARKS) FROM STUDENTS);
```

MIN - Minimum value:

```
SELECT MIN(AGE) AS min_age FROM STUDENT;  
SELECT MIN(MARKS) AS lowest_marks FROM STUDENTS;
```

3.3 GROUP BY and HAVING

GROUP BY - Group records by column:

```
SELECT BRANCH_CODE, COUNT(*) AS num_students  
FROM STUDENT GROUP BY BRANCH_CODE;
```

```
SELECT BRANCH_CODE, AVG(MARKS) AS avg_marks  
FROM STUDENTS GROUP BY BRANCH_CODE;
```

HAVING - Filter grouped data:

```
SELECT BRANCH_CODE, COUNT(*) AS num_students  
FROM STUDENT GROUP BY BRANCH_CODE HAVING COUNT(*) > 2;
```

```
SELECT BRANCH_CODE, AVG(MARKS) AS avg_marks  
FROM STUDENTS GROUP BY BRANCH_CODE HAVING AVG(MARKS) > 60;
```

4. SCALAR FUNCTIONS

Scalar functions operate on single values and return single result.

4.1 String Functions

UCASE/UPPER - Convert to uppercase:

```
SELECT UCASE(NAME) FROM STUDENT;  
SELECT UPPER(NAME) AS name_upper FROM STUDENT;
```

LCASE/LOWER - Convert to lowercase:

```
SELECT LCASE(NAME) FROM STUDENT;  
SELECT LOWER(NAME) AS name_lower FROM STUDENT;
```

MID/SUBSTR - Extract substring:

```
SELECT MID(NAME, 1, 3) FROM STUDENT; -- First 3 characters  
SELECT SUBSTR(NAME, 2, 4) FROM STUDENT; -- From position 2, length 4
```

LEN/LENGTH - String length:

```
SELECT LEN(NAME) FROM STUDENT;  
SELECT LENGTH(NAME) AS name_length FROM STUDENT;
```

CONCAT - Concatenate strings:

```
SELECT CONCAT(NAME, ' - ', BRANCH_CODE) FROM STUDENT;  
SELECT NAME || ' studies ' || BRANCH_CODE FROM STUDENT; -- Using ||
```

TRIM - Remove leading/trailing spaces:

```
SELECT TRIM(NAME) FROM STUDENT;  
SELECT LTRIM(NAME) FROM STUDENT; -- Remove left spaces  
SELECT RTRIM(NAME) FROM STUDENT; -- Remove right spaces
```

REPLACE - Replace characters:

```
SELECT REPLACE(NAME, 'A', 'X') FROM STUDENT;
```

4.2 Numeric Functions

ABS - Absolute value:

```
SELECT ABS(-5) AS absolute_value;  
SELECT ABS(MARKS - 50) FROM STUDENTS;
```

ROUND - Round to decimal places:

```
SELECT ROUND(MARKS, 0) FROM STUDENTS; -- Round to nearest integer  
SELECT ROUND(3.14159, 2); -- Result: 3.14
```

CEIL/CEILING - Round up:

```
SELECT CEIL(3.2); -- Result: 4  
SELECT CEILING(MARKS / 10 * 10) FROM STUDENTS;
```

FLOOR - Round down:

```
SELECT FLOOR(3.9); -- Result: 3
```

POWER/POW - Exponentiation:

```
SELECT POWER(2, 3); -- Result: 8  
SELECT POW(MARKS, 2) FROM STUDENTS; -- Square of marks
```


SQRT - Square root:

```
SELECT SQRT(16); -- Result: 4
SELECT SQRT(MARKS) FROM STUDENTS;
```

MOD - Modulo (remainder):

```
SELECT MOD(10, 3); -- Result: 1
SELECT MOD(ROLL_NO, 2) FROM STUDENT;
```

4.3 Date Functions

NOW/CURRENT_DATE - Current date and time:

```
SELECT NOW();
SELECT CURRENT_DATE;
SELECT CURRENT_TIMESTAMP;
```

DATE - Extract date part:

```
SELECT DATE(NOW()); -- Today's date only
```

YEAR, MONTH, DAY - Extract date components:

```
SELECT YEAR(DATE_OF_BIRTH) FROM STUDENT;
SELECT MONTH(DATE_OF_BIRTH) FROM STUDENT;
SELECT DAY(DATE_OF_BIRTH) FROM STUDENT;
```

DATE_ADD - Add days/months/years:

```
SELECT DATE_ADD(NOW(), INTERVAL 10 DAY);
SELECT DATE_ADD(NOW(), INTERVAL 1 MONTH);
```

DATEDIFF - Difference between dates:

```
SELECT DATEDIFF(NOW(), DATE_OF_BIRTH) FROM STUDENT;
```

DATE_FORMAT - Format date:

```
SELECT DATE_FORMAT(NOW(), '%Y-%m-%d'); -- 2024-03-14
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y'); -- 14/03/2024
```

5. PRACTICAL QUERY EXAMPLES

5.1 Student Management System Queries

Find all students in CS branch

```
SELECT * FROM STUDENT WHERE BRANCH_CODE = 'CS';
```

Count students per branch

```
SELECT BRANCH_CODE, COUNT(*) as count FROM STUDENT GROUP BY BRANCH_CODE;
```

Find student with highest marks

```
SELECT NAME, MAX(MARKS) FROM STUDENTS;
```

List students with age between 18-20

```
SELECT * FROM STUDENT WHERE AGE BETWEEN 18 AND 20;
```

Find average marks per branch

```
SELECT BRANCH_CODE, AVG(MARKS) FROM STUDENTS GROUP BY BRANCH_CODE;
```

Students in CS branch with marks > 70

```
SELECT * FROM STUDENTS WHERE BRANCH_CODE='CS' AND MARKS > 70;
```

Count how many students in each branch

```
SELECT BRANCH_CODE, COUNT(*) FROM STUDENT GROUP BY BRANCH_CODE HAVING COUNT(*) > 0;
```

Get students sorted by marks (descending)

```
SELECT * FROM STUDENTS ORDER BY MARKS DESC;
```

Find student name and branch name

```
SELECT s.NAME, b.BRANCH_NAME FROM STUDENT s INNER JOIN BRANCH b ON s.BRANCH_CODE = b.BRANCH_CODE;
```

Update student age to 21

```
UPDATE STUDENT SET AGE = 21 WHERE ROLL_NO = 1;
```

Delete students with age < 18

```
DELETE FROM STUDENT WHERE AGE < 18;
```

QUICK SQL REFERENCE

Comparison Operators

Operator	Meaning
=	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
BETWEEN	Within range
IN	Match any in list
LIKE	Pattern matching
IS NULL	Is empty

Logical Operators

Operator	Meaning	Example
AND	All conditions must be true	WHERE age > 18 AND branch_code = 'CS'
OR	At least one condition is true	WHERE branch = 'CS' OR branch = 'IT'
NOT	Negates a condition	WHERE NOT branch_code = 'CS'

LIKE Pattern Matching

Pattern	Meaning
%	Zero or more characters
_	Exactly one character
'R%'	Starts with R
'%H'	Ends with H
'%AM%'	Contains AM
'_A%'	Second character is A